

The propositions package*

Cian Dorr (with help from Claude Code)
ciandorr@gmail.com

2026/02/13

Abstract

The `propositions` package provides a key-value driven system for labelling propositions, theses, and premises in academic papers. Items may be given names like ‘(P)’ or ‘Physicalism’, or auto-numbered using different counters; all carry robust cross-references with configurable formatting. The package integrates with `amsmath`, `hyperref`, and `cleveref`.

Contents

1	Introduction	2
2	History	2
3	Basic usage	2
4	Advanced usage	3
5	The prop environment and <code>\pitem</code>	4
6	Item types	7
6.1	Built-in types	8
7	Cross-referencing	11
7.1	How it works: <code>\propapply</code>	13
8	Package options	13
8.1	List dimensions	14
8.2	Default types	14
8.3	Formatting and referencing equation numbers	15
9	Compatibility	15
10	Known issues	16

*This document describes version v0.2, dated 2026/02/13.

1 Introduction

In some academic disciplines (such as philosophy), it is common to have displayed propositions (examples, theses, premises, . . .) with various kinds of labels. A thesis might be referred to as ‘(P)’ or ‘Physicalism’; the premises of an argument might be numbered as ‘P1’, ‘P2’, ‘P3’, . . . ; or examples might be numbered consecutively over the course of a whole article. Standard L^AT_EX environments like `enumerate` can handle some of these cases, but cross-referencing is awkward: `\ref` produces a bare number or letter, and the author must manually add parentheses or other formatting at every point of reference. The standard `description` environment does not allow cross-referencing at all.

The `propositions` package solves this by attaching formatting information to each label. A short item like `\pitem[P]` is displayed as “(P)” and `\ref` automatically produces “(P)” as well—complete with parentheses, hyperlinks, and `cleveref` support. The full key-value interface supports named items, numbered items, custom counters, glosses, shorthands, and per-item format overrides.

The `\ptag` command (which requires `amsmath`) extends this to displayed math environments: an equation can be tagged with a proposition label instead of (or using) its equation number.

2 History

I wrote the ancestor to this package in the 90s while finishing my Ph.D. thesis, but never documented it or shared it with the world. This new version is a thorough re-implementation in L^AT_EX3, written in 2026 with extensive help from Claude Code. I hope others will find it as useful as I have.

3 Basic usage

Load the package with `\usepackage{propositions}` or `\usepackage[options]{propositions}` (see [section 8](#) below for valid package options).

The `prop` environment generates a list of propositions, each introduced by a `\pitem`. `\pitem` with an optional argument gives a `description`-like label:

<code>\begin{prop}</code>	
<code>\pitem[Physicalism] Everything is</code>	Physicalism Everything is physi-
<code>physical. \label{phys}</code>	cal.
<code>\pitem[Idealism] Everything is</code>	Idealism Everything is mental.
<code>mental. \label{ideal}</code>	
<code>\end{prop}</code>	

Unlike the standard `description` environment, one can refer back to these propositions using the standard `\ref` command:

<code>\ref{phys}</code> is more plausible	Physicalism is more plausible than
than <code>\ref{ideal}</code> .	Idealism .

With no optional argument, `\pitem` will by default generate numbered items similar to `enumerate`, but with numbering that persists across the document:

<pre> \begin{prop} \pitem Every atom is physical. \label{atoms} \end{prop} \ref{phys} follows from the conjunction of \ref{atoms} and \begin{prop} \pitem Everything is an atom. \label{atomism} \end{prop} </pre>	(1) Every atom is physical. Physicalism follows from the conjunction of (1) and (2) Everything is an atom.
--	--

As with `enumerate`, the counter and formatting depend on the nesting level:

<pre> \begin{prop} \pitem \label{dual} \begin{prop} \pitem Some things are physical. \label{some} \pitem Some things are not physical. \label{notall} \end{prop} \end{prop} </pre>	(3) a. Some things are physical. b. Some things are not physical. Without (3a), (3) would be consistent with Idealism.
--	--

Without `\ref{some}`, `\ref{dual}` would be consistent with `\ref{ideal}`.

4 Advanced usage

The format of the proposition labels, and of subsequent references, are both configurable using a key=value syntax (see section 5 for the possible keys):

<pre> \begin{prop} \pitem[No Overlap, align=flush, display format=\textsc{#1}, ref format=\textit{#1}] Nothing mental is physical. \label{incomp} \end{prop} </pre>	NO OVERLAP Nothing mental is physical. Is <i>No Overlap</i> consistent with Physicalism?
---	---

Is `\ref{incomp}` consistent with `\ref{phys}`?

Preset *item types* can be declared and used instead of configuring the `\pitems` one at a time:

<pre> \begin{prop} \pitem[Nihilism, type=thesis] There is nothing. \label{nihilism} \end{prop} </pre>	NIHILISM There is nothing. Does Nihilism imply Physicalism, (3), or both? Discuss.
---	--

Does `\ref{nihilism}` imply `\ref{phys}`, `\ref{dual}`, or both? Discuss.

Shortcut commands—e.g., `\titem[⟨keys⟩]` for `\pitem[type=thesis, ⟨keys⟩]`—can also be defined. The package loads with a range of predefined types.

When the package is loaded with `\usepackage[equations]{propositions}`, first-level `\pitems` use the same counter as equations. (This looks better with the `leqno` option to `\documentclass`.)

<code>\begin{equation}</code>	
<code>\exists x (\text{Mental}(x)</code>	
<code>\wedge \text{Physical}(x))</code>	(4) $\exists x(\text{Mental}(x) \wedge \text{Physical}(x))$
<code>\end{equation}</code>	
<code>\begin{prop}</code>	(5) There is overlap between the
<code>\pitem</code>	mental and the physical.
<code>There is overlap between</code>	
<code>the mental and the physical.</code>	
<code>\end{prop}</code>	

The `\ptag` command (requires `amsmath`) is an analogue of `\pitem` that works inside displayed math environments.

<code>\begin{equation}</code>	
<code>\ptag[Monism] \label{mon}</code>	
<code>\exists x \forall y (y = x)</code>	Monism $\exists x \forall y (y = x)$
<code>\end{equation}</code>	
Is <code>\ref{mon}</code> compatible with	Is Monism compatible with (3)?
<code>\ref{dual}</code> ?	

5 The prop environment and \pitem

```
\begin{prop}[\langle keys \rangle]
  \langle environment content \rangle
\end{prop}
```

Creates a displayed list of propositions. It is a standard L^AT_EX list, so by default its formatting will depend on the standard length parameters like `\itemsep` and `\topsep`, although these can be overridden by setting package keys.

Within `prop`, `\pitem` (see below) creates labelled items. The ordinary `\item` command is still available for unlabelled items.

Any display math environment (`\equation`, `align`, etc.) used inside `prop` or `inlineprop` automatically increments the nesting level for its duration, so that `\ptag` inside such an environment behaves as if it were one level deeper. The optional `\langle keys \rangle` argument accepts the same keys as `\propoptions`^{→ P. 13} (section 8), with effects local to this environment.

```
\begin{inlineprop}[\langle keys \rangle]
  \langle environment content \rangle
\end{inlineprop}
```

Like `prop`, but does not create a list. Allows `\pitem` to be used outside list environments, e.g. for generating numbers at the beginning of paragraphs. Steps the `prop` counter and increments the nesting level. Accepts the same optional `\langle keys \rangle` as `prop`.

```
\pitem[\langle keys \rangle]
```

Inside the `prop` and `inlineprop` environments, introduces a labelled proposition. The optional argument is a comma-separated list of `\langle key \rangle = \langle value \rangle` pairs.

When used without an optional argument (or without setting `name`, `counter`, or `type`), the type is determined by the `default nameless type` key (section 8). By default this is `numbered`, which dispatches by nesting depth to `levelone`, `leveltwo`, ... via `\proplevelchoice`.

`\ptag[⟨keys⟩]`

(Available only when `amsmath` is loaded.) Works inside displayed math environments like `equation` and `align`. Accepts the same keys as `\pitem`^{P.4}, except that `align` has no effect (since positioning is controlled by the tag placement system).

The following keys can be used in the optional argument of `\pitem` or the argument of `\ptag`:

`name=⟨text⟩` (no default)

The proposition's name. A bare string (without `=`) in the key list is equivalent to `name=⟨text⟩`.

`type=⟨type⟩` (no default)

An item type, equivalent to a preset collection of keys. Types can be declared using `\SetItemType`^{P.7} or `\DeclareNumberedType`^{P.7}, and several come predefined (section 6).

`counter=⟨name⟩` (no default)

Counter to use. The counter is automatically stepped, and the item's `name` is set to `\the⟨name⟩` (though this can be overridden by `counter format` or `name`).

`counter format=⟨template⟩` (no default)

How to display the counter value. Use `#1` for the counter name, e.g. `counter format=\roman{#1}`. When set, this is used instead of `\the⟨counter⟩` for generating the item's name.

`format=⟨template⟩` (no default)

Formatting applied to the `name`: use `#1` for the argument, e.g. `format=\textbf{(#1)}`. Shorthand for setting both `display format` and `ref format`.

`display format=⟨template⟩` (no default)

Format for displaying the name in the proposition's label. Does not affect cross-references.

`label format=⟨template⟩` (no default)

Format applied to the *entire* assembled label, i.e. the name (after `display format`), shorthand, and gloss together. Use `#1` for the whole assembled text. For example, `label format=#1:` appends a colon after the complete label. Does not affect cross-references.

`ref format=⟨template⟩` (no default)

Format for subsequent cross-references to this proposition. Does not affect the display.

shorthand= $\langle text \rangle$ (no default)
 An abbreviation displayed after the name. If present, the shorthand becomes the reference text: `\ref` produces the shorthand (formatted with `ref format`) rather than the full name.

shorthand format= $\langle template \rangle$ (initially $\sim \text{[#1]}$)
 Format for displaying the shorthand in the label.

gloss= $\langle text \rangle$ (no default)
 A parenthetical gloss displayed after the name. Does not affect cross-references.

gloss format= $\langle template \rangle$ (initially $\sim \text{(#1)}$)
 Format for displaying the gloss in the label.

ref= $\langle text \rangle$ (no default)
 Explicitly set the reference text, overriding what would be derived from `name`, `counter`, or `shorthand`.

label= $\langle label \rangle$ (no default)
 Equivalent to a trailing `\label{\langle label \rangle}`.

crefname= $\langle type \rangle$ (no default)
 When `cleveref` is loaded, assigns an arbitrary reference type to this proposition. For example, `crefname=lemma` causes `\cref` to use the names defined by `\crefname{lemma}{...}{...}` instead of the default `proposition` type. The $\langle type \rangle$ must be known to `cleveref`; new types can be declared with `\crefname`.

align= $\langle type \rangle$ (no default)
 How the label should be positioned. Possible values: `left` (standard left-aligned label, offset controlled by `\labelwidth` and `\labelsep`), `right` (right-aligned, like `enumerate`), `center` (centered within the label box), `flush` (aligned with left margin of item text), `nextline` (label on its own line), and `flush-nextline`. Has no effect inside `inlineprop` or `\ptag`. For left-margin alignment, use `labelindent=0em` instead.

labelindent= $\langle length \rangle$ (no default, per-item)
 Positions the label's left edge at $\langle length \rangle$ from the enclosing left margin. For example, `labelindent=0em` aligns the label with the surrounding text. When the label plus `\labelsep` would overflow into the item text, the first line is pushed right automatically (like the global `labelindent` key). Can be set per-item, per-type, or globally via `\proptoptions`.

labelwidth= $\langle length \rangle$ (no default, per-item)
 Overrides the effective label box width for this item. The label is repositioned as if `\labelwidth` were $\langle length \rangle$, without changing the actual list dimension.

labelsep= $\langle length \rangle$ (no default, per-item)
 Overrides the effective label-to-text gap for this item.

itemindent= $\langle length \rangle$ (no default, per-item)
 Overrides the effective first-line indent for this item. Only increases are fully supported; decreases are clamped to zero.

6 Item types

`\SetItemType{⟨name⟩}{⟨keys⟩}`

Defines or modifies an item type for use with the `type` key. All `\pitem`^{P.4} keys are accepted, plus the following:

`type=⟨parent⟩` (no default)

Inherit from a parent type. When an item is created, the parent's settings are loaded first (recursively, if the parent itself has a parent), then this type's own keys are applied on top.

The argument may be a macro which is expanded when the item is created: for example, a type with `type=\{⟨type1⟩\},\{⟨type2⟩\},\{⟨type3⟩\}` will resolve to `\{⟨type1⟩\}`, `\{⟨type2⟩\}`, `\{⟨type2⟩\}` depending on the nesting depth. The built-in `numbered` and `eqnum` types use this mechanism.

`macro=⟨command⟩` (no default)

A new user macro, equivalent to `\pitem[type=⟨name⟩]`. Any further keys given to the macro are passed to `\pitem`.

If the type `⟨name⟩` already exists, `\SetItemType` modifies or adds keys. For example, `\SetItemType{proposition}{align=flush}` changes the alignment of the built-in `proposition` type while preserving its other settings.

<pre> \SetItemType{angle}{ labelindent = 0em, display format = \textbf{⟨\rangle\$#1\$\rangle\$}, ref format = ⟨\rangle\$#1\$\rangle\$, macro = \angitem } \begin{prop} \angitem[Angle thesis] Everything is angular. \end{prop} No further discussion of \lastref{} is needed.</pre>	⟨Angle thesis⟩ Everything is angular. No further discussion of ⟨Angle thesis⟩ is needed.
--	---

`\DeclareNumberedType{⟨name⟩}[⟨keys⟩]`

Creates a new L^AT_EX counter named `⟨name⟩` and a matching item type with `counter=⟨name⟩`. All `\SetItemType` keys are accepted, plus:

`parent=⟨counter⟩` (no default)

A parent counter; the new counter resets when the parent steps (same mechanism as `\numberwithin`). A dedicated `prop` counter (stepped by each `prop` and `inlineprop`) is available for non-persistent numbering.

<code>\DeclareNumberedType{P}</code>	P0 First premise.
<code>\begin{prop}</code>	
<code>\pitem[counter=P] First premise.</code>	P0 Second premise.
<code>\label{p1}</code>	From P0 and P0 ...
<code>\pitem[counter=P] Second premise.</code>	
<code>\label{p2}</code>	
<code>\end{prop}</code>	
<code>From \ref{p1} and \ref{p2}\ldots</code>	

`\proplevelchoice{⟨item1, item2, ...⟩}`

Picks the entry at the current nesting depth (1-indexed, clamped to the list length). At depth 0 (outside any `prop` environment), returns the first entry. This macro is fully expandable and can be used in the `name`, `format`, and `type` keys of `\SetItemType` to create depth-dependent types.

6.1 Built-in types

The following item types are predefined. Each entry shows the defining code (using user-facing commands) and a live example. All types can be modified with `\SetItemType` ^{P.7}.

Text types

plain Unformatted text label.

<code>\SetItemType{plain}{format = #1}</code>	
---	--

Claim An unformatted item.

proposition Bold text label. Auto-selected when `\pitem` has a name but no `type`.

<code>\SetItemType{proposition}{format = \textbf{#1}}</code>	
--	--

Thesis A bold-formatted item.

thesis Small-caps name at the text margin (flush alignment). Shortcut: `\titem`.

<code>\SetItemType{thesis}{format = \textsc{#1},</code> <code>align = flush, macro = \titem}</code>	
--	--

HYPOTHESIS A flush-aligned item.

vignette Italic name at the text margin with a colon in the label.

<code>\SetItemType{vignette}{ref format = \textit{#1},</code> <code>align = flush, label format = \textit{#1:}}</code>	
---	--

Example: A vignette item.

bullet Bullet symbol varying by depth (like `\itemize`). Shortcut: `\bitem`.

```
| \SetItemType{bullet}{align = center,  
|   name = \proplevelchoice{\textbullet,  
|     {\normalfont\textendash}, \textasteriskcentered,  
|     \textperiodcentered},  
|   display format = #1, macro = \bitem}
```

- A bullet item.

Numbered types with dedicated counters

roman Roman-numeral counter, resets per section. Shortcut: `\ritem`.

```
| \DeclareNumberedType{roman}[parent = section,  
|   counter format = \roman{#1}, format = (#1), macro = \ritem]
```

- (i) A roman-numbered item.

alph Alphabetic counter, resets per section. Shortcut: `\aitem`.

```
| \DeclareNumberedType{alph}[parent = section,  
|   counter format = \alph{#1}, format = (#1), macro = \aitem]
```

- (a) An alphabetic item.

Level-specific types

These types are used by the `numbered` and `eqnum` dispatcher types (below). They share the counters `numpropi`–`numpropv`, which reset automatically when a `\pitem` at the next lower level is processed.

levelone Level-1 numbered items: arabic in parentheses.

```
| \SetItemType{levelone}{counter = numpropi, format = (#1)}
```

leveltwo Level-2: lowercase alphabetic with dot.

```
| \SetItemType{leveltwo}{counter = numpropii,  
|   display format = #1., ref format = \parentref{#1}}
```

levelthree Level-3: lowercase roman in parentheses.

```
| \SetItemType{levelthree}{counter = numpropiii,  
|   display format = (#1), ref format = \parentref{.#1}}
```

levelfour Level-4: uppercase alphabetic with dot.

```
| \SetItemType{levelfour}{counter = numpropiv,  
|   display format = #1., ref format = \parentref{#1}}
```

`levelfive` Level-5: uppercase roman in parentheses.

```
| \SetItemType{levelfive}{counter = numpropv,  
|   display format = (#1), ref format = \parentref{.#1}}
```

`equation` Uses the equation counter with customisable equation tag formatting.

```
| \SetItemType{equation}{counter = equation,  
|   display format = <eqdispfmt>, ref format = <eqreffmt>}
```

(The display and ref formats use internal hooks controlled by the `equation` format keys; see [section 8](#).)

Dispatcher types

These types use `type` with `\proplevelchoice` to dispatch to a level-specific type at resolution time.

`numbered` The default nameless type. Dispatches to `levelone`–`levelfive` by nesting depth.

```
| \SetItemType{numbered}{type = \proplevelchoice{  
|   levelone, leveltwo, levelthree, levelfour, levelfive}}
```

`eqnum` Like `numbered`, but uses `equation` at level 1. Activated by the `equations` package option (which sets default nameless type = `eqnum`).

```
| \SetItemType{eqnum}{type = \proplevelchoice{  
|   equation, leveltwo, levelthree, levelfour, levelfive}}
```

`hierarchical` Produces 1, 1.1, 1.1.1... numbering using `\parentref` and `counter` format. Defined via two helper types:

```
| \SetItemType{h-base}{counter = numpropi,  
|   counter format = \arabic{#1}, format = #1}  
| \SetItemType{h-sub}{  
|   counter = \proplevelchoice{numpropi, numpropii,  
|     numpropiii, numpropiv, numpropv},  
|   counter format = \arabic{#1},  
|   format = \parentref{.#1}}  
| \SetItemType{hierarchical}{type = \proplevelchoice{  
|   h-base, h-sub}}
```

7 Cross-referencing

Labels placed after `\pitem` items work with the standard `\label/\ref` mechanism. The key difference from ordinary $\text{\LaTeX}$ references is that `\ref` produces *formatted* output: for example, `\textbf` might be applied to the name, or the number might be wrapped in parentheses. The formatting is controlled by the `format` key (or separately by `display format` and `ref format`).

`\Ref{\label}`
`\Ref*{\label}`

Titlecase variants of `\ref` and `\ref*`: uppercases the first letter of the formatted output. (For example, if `\ref{thesis}` produces ‘the Identity Theory’, then `\Ref{thesis}` produces ‘The Identity Theory’.) The starred form suppresses the hyperlink. Note: `\Ref` only affects references produced by this package (which are stored via `\propapply`); for other references, it behaves like `\ref`.

`\nref{\label}`
`\nref*{\label}`
`\Nref{\label}`
`\Nref*{\label}`

“Naked ref.” Outputs the bare reference content with all formatting stripped. If `\ref{premise}` produces ‘(P1)’, then `\nref{premise}` produces ‘P1’. The starred form suppresses the hyperlink. `\Nref` is the titlecase variant: it uppercases the first letter of the bare content.

`\nref` is often useful in the argument of `\pitem`, when the the name of one proposition should depend on that of another:

<pre>\SetItemType{default}{format=(#1)} \begin{prop} \pitem[Phys] Everything is physical. \label{phys2} \pitem[\nref{phys2}*] Almost everything is physical. \label{newphys2} \pitem[\ref{premise}*] Note that the reference in the name of proposition \ref{newphys2} is made using \ref instead of \nref. \end{prop}</pre>	Phys Everything is physical. Phys* Almost everything is physical. 7* Note that the reference in the name of proposition (Phys*) is made using <code>\nref</code>: using <code>\ref</code> instead would have produced two sets of parentheses, one from the reference and one applied by the display format.
--	---

Warning: documents where the name of one item includes a reference to that of another, and there are further references to that item, will require multiple $\text{\LaTeX}$ runs to resolve all references. To save time, it is better to avoid long chains of dependencies of this sort.

`\oref[<prefix>][<suffix>]{<label>}`
`\oref*[<prefix>][<suffix>]{<label>}`
`\Oref[<prefix>][<suffix>]{<label>}`
`\Oref*[<prefix>][<suffix>]{<label>}`

“Ref with options.” Extends `\ref` by injecting a prefix and/or suffix *inside* the formatting. With one optional argument, `<suffix>` is appended;

with two, $\langle prefix \rangle$ is prepended and $\langle suffix \rangle$ appended. For instance, if $\backslash\text{ref}\{\text{premise}\}$ produces ‘(P1)’, then $\backslash\text{oref}[*]\{\text{premise}\}$ produces ‘(P1*)’ and $\backslash\text{oref}[\text{old}\sim][*]\{\text{premise}\}$ produces ‘(old~P1*)’.

$\backslash\text{Oref}$ is the titlecase variant; the starred forms suppress the hyperlink.

$\backslash\text{oref}$ can also be useful in the name of $\backslash\text{pitems}$, if one wants the display format for the modified item to depend on that originally used

```
\begin{prop}
  \label{premise}
  \pitem[type=plain,name=\oref[*]{phys2}]
  This will use boldface and
  parentheses because the original
  referenced item did.
\end{prop}
```

(Phys*) This will use boldface and parentheses because the original referenced item did.

Another handy use for $\backslash\text{oref}$ is in combination with $\backslash\text{nref}$ to refer to ranges:

```
The first two numbered examples in
this document were
were \oref[--\nref{atomism}]{atoms}.
```

The first two numbered examples in this document were (1–2).

$\backslash\text{lastref}[\langle prefix \rangle]\{\langle suffix \rangle\}$

$\backslash\text{Lastref}[\langle prefix \rangle]\{\langle suffix \rangle\}$

Formatted reference to the most recently processed $\backslash\text{pitem}$ or $\backslash\text{ptag}$, even without a $\backslash\text{label}$. Useful for back-references in running text. With one argument, $\langle suffix \rangle$ is appended; with two, $\langle prefix \rangle$ is also prepended. Use $\backslash\text{lastref}\{\}$ for a plain reference. $\backslash\text{Lastref}$ is the titlecase variant: it uppercases the first letter.

$\backslash\text{nLastref}$

Like $\backslash\text{lastref}\{\}$ but returns the bare content without formatting. Takes no arguments; simply output any desired suffix directly afterwards.

$\backslash\text{parentref}[\langle prefix \rangle]\{\langle suffix \rangle\}$

$\backslash\text{Parentref}[\langle prefix \rangle]\{\langle suffix \rangle\}$

Inside a nested `prop` (or `inlineprop`), produces a formatted reference to the most recent item of the enclosing level. Same argument convention as $\backslash\text{lastref}$. $\backslash\text{Parentref}$ is the titlecase variant: it uppercases the first letter.

$\backslash\text{nParentref}$

Like $\backslash\text{parentref}\{\}$ but returns the bare content without formatting. Takes no arguments; simply output any desired suffix directly afterwards.

$\backslash\text{parentref}$ and $\backslash\text{nParentref}$ are useful for making subitems whose names derive from their parent’s:

```

\DeclareNumberedType{inner}[
  counter format=\alph{inner},
  display format=#1.]
\begin{prop}
  \pitem[OI] Outer item.
  \label{outer2}
  \begin{prop}
    \pitem[type=inner,
      format=\parentref{.#1}]
    \label{dsub1}
    Ref: \ref{dsub1},
    naked: \nref{dsub1}.
    \pitem[type=inner,
      display format=#1.,
      ref format=\parentref{.#1}]
    \label{dsub2}
    Ref: \ref{dsub2},
    naked: \nref{dsub2}.
  \end{prop}
\end{prop}

```

OI Outer item.

OI.a Ref: (OI.a), naked: a.

b. Ref: (OI.b), naked: b.

The built-in `leveltwo` and `levelthree` types have `ref format=\parentref{#1}` and `ref format=\parentref{.#1}`, respectively, so that if the parent references as ‘(P1)’, a `leveltwo` sub-item references as ‘(P1a)’ and `\nref` returns just ‘a’.

7.1 How it works: `\propapply`

`\propapply{<template>}{<content>}`

Internally, each reference is stored in the `.aux` file as `\propapply{<template>}{<content>}`.

The `<template>` contains formatting with the placeholder `\propfmtarg` where content appears. At reference time, `\propapply` evaluates the template with `\propfmtarg` bound to `<content>`. The `\oref`^{P.11} and `\nref`^{P.11} commands work by locally redefining `\propapply`.

In normal use, you need not interact with `\propapply` directly.

`\propfmtarg`

Placeholder used inside templates; expands to the content argument of the enclosing `\propapply`.

8 Package options

`\proppositions{<keys>}`

Sets package-level keys. These can also be set:

- In the optional argument of `prop` and `inlineprop` (local to that environment).
- In the preamble with `\usepackage[<options>]{propositions}` (global).

Exception 1: keys containing # (such as `equation format`) cannot be set in the optional argument of `\usepackage`, due to how L^AT_EX handles # in option values.

Exception 2: `equations` is a global key; it cannot be used in the optional argument of `prop` or `inlineprop`.

8.1 List dimensions

`topsep`= $\langle \text{length} / \text{length list} \rangle$
`partopsep`= $\langle \text{length} / \text{length list} \rangle$
`itemsep`= $\langle \text{length} / \text{length list} \rangle$
`parsep`= $\langle \text{length} / \text{length list} \rangle$
`leftmargin`= $\langle \text{length} / \text{length list} \rangle$
`rightmargin`= $\langle \text{length} / \text{length list} \rangle$
`labelwidth`= $\langle \text{length} / \text{length list} \rangle$
`labelsep`= $\langle \text{length} / \text{length list} \rangle$
`itemindent`= $\langle \text{length} / \text{length list} \rangle$
`listparindent`= $\langle \text{length} / \text{length list} \rangle$

Override the standard L^AT_EX list dimensions. Accept the same values as `\setlength`, including rubber lengths (e.g. `itemsep=4pt plus 2pt`). Each key may also take a comma-separated list of per-level values: e.g. `leftmargin={2.5em, 0em}`. The first value applies at level 1, the second at level 2, etc. Gaps in the list (e.g. `leftmargin={, 0em}`) cause the class default to be used for that level.

`labelindent`= $\langle \text{length} / \text{length list} \rangle$ (no default)

Positions label left edges at $\langle \text{length} \rangle$ from the enclosing margin, adjusting `\labelsep` or `\itemindent` as needed. Positive values move rightward, negative leftward.

`tightspacing` (no value)

Sets all vertical spacing to the compact defaults that the standard document classes use for level-three lists (`topsep` and `itemsep` to 2pt with stretch/shrink, `parsep` to 0pt, `partopsep` to 1pt). Individual dimension keys set afterward override.

`nosep` (no value)

Sets `\topsep`, `\itemsep`, and `\parsep` all to zero.

`continue`= $\langle \text{boolean} \rangle$ (initially `true`)

When `true` (the default), a `prop` environment that immediately follows another `prop` (with no intervening paragraph text) replaces the normal `\topsep`-based inter-list space with `\itemsep + \parsep`, giving the visual appearance of a single continuous list. This works at any nesting level. Set to `false` to suppress this behaviour for a specific environment or globally via `\propoptions`^{→ P. 13}.

8.2 Default types

When `\pitem` or `\ptag` is used without an explicit `type`, one of the following defaults is selected based on whether a name/counter was given and whether `ptag` mode is active.

`default type`= $\langle \text{type} \rangle$ (initially `proposition`)

The type used when a `name` or `counter` is given but no explicit `type`.

`default ptag type=<type>` (initially empty)
 If set, overrides `default type` for `\ptag` items only.

`default nameless type=<type>` (initially `numbered`)
 The type used when `\pitem` or `\ptag` has no optional argument (no name, counter, or type). The built-in `numbered` type dispatches to `levelone...levelfive` by nesting depth via `\proplevelchoice` (see [subsection 6.1](#)).

`default ptag nameless type=<type>` (initially empty)
 If set, overrides `default nameless type` for `\ptag` items only.

8.3 Formatting and referencing equation numbers

`equations` (no value, **global only**)
 An alias for `default nameless type=eqnum`. The built-in `eqnum` type is like `numbered` but uses the `equation` counter at nesting level 1, sharing counter values with standard displayed equations. Also installs the equation format hooks (see below), so that `\oref` and `\nref` work with equation labels.

`equation format=<template>` (initially `#1`)
 Shorthand: sets both `equation display format` and `equation ref format`. Also installs the equation format hooks independently of the `equations` option.

`equation display format=<template>` (initially `#1`)
 Controls how equation tags appear in the PDF (via `\tagform@`). Use `#1` for the number. Does not affect `\ref` output. Also installs the equation format hooks independently of the `equations` option. Locally scoped.

`equation ref format=<template>` (initially `#1`)
 Controls how `\ref` (and `\oref`, `\nref`) render equation labels. Use `#1` for the number. Does not affect the displayed tag. Also installs the equation format hooks independently of the `equations` option. Locally scoped.

9 Compatibility

The `propositions` package is designed to work with `hyperref`, `cleveref`, and `amsmath`. `amsmath` is required for `\ptag` and the `equations` option. With `cleveref` loaded, all `\pitems` are assigned to a default `proposition` reference type; the `crefname` key can override this.

Recommended load order:

```
\usepackage{hyperref}
\usepackage{amsmath} % if using \ptag
\usepackage{propositions}
\usepackage{cleveref} % if used
```

10 Known issues

When using `\ptag` with a named counter (e.g. `\ptag[counter=P]`) inside an `ams-math` equation environment, `hyperref` may emit warnings of the form:

```
pdfTeX warning: destination with the same  
identifier (name{equation.N}) has been already  
used, duplicate ignored
```

These warnings are harmless and do not affect the correctness of cross-references.